

PROGRAMOWANIE ZAAWANSOWANYCH SERWISÓW INTERNETOWYCH

Scenariusz projektu obiektowego

Zbudujemy prosty serwis internetowy, który będzie realizował operacja **CRUD** (create-read-update-delete) dla wybranej tabeli bazy danych. U nas jednak operacja **delete** nie będzie usuwała rekordu w bazie danych. Zamiast tego będzie on jedynie dezaktywowany, umieszczany w koszu.

Zacznijmy od uruchomienia programu **XAMPP** i włączmy moduły **Apache** oraz **MySQL**. Sprawdźmy czy działają.



Otwórzmy folder **htdocs** znajdujący się w folderze instalacyjnym XAMPP i stwórzmy w nim folder **phpadvanced** dla naszego projektu. W nim będziemy umieszczać wszystkie pliki. Zacznijmy od stworzenia pliku **index.php** i umieścimy w nim na razie kod wypisujący na ekranie tekst **Hello world!**.

```
1 <?php
2 echo "Hello world!";
```

Otwórzmy teraz przeglądarkę i wpiszmy adres **localhost/phpadvanced**. Widzimy napis **Hello world!?** Czyli wszystko działa! Możemy rozpocząć właściwą pracę nad naszym serwisem.

W jaki sposób będzie działał nasz serwis? Stworzymy klasę abstrakcyjną **Page**. Następnie dla każdej tabeli w bazie danych, dla której będziemy chcieli wykonywać operacje **CRUD** stworzymy odpowiednią klasę dziedziczącą po **Page** i wypełnimy metody abstrakcyjne. Naszą pierwszą tabelą na której będziemy pracować będzie **InternalEvents** – wydarzenia wewnętrzne. W dalszej części będziemy używali pojęcia **Model**. Będzie ono oznaczać jedno konkretne wydarzenie wewnętrzne.

Zacznijmy więc od stworzenia bazy danych. Otwórzmy **phpMyAdmin** (wpisując w przeglądarce adres **localhost/phpmyadmin**), stwórzmy bazę danych i nazwijmy ją **phpadvanced** i używając poniższego skryptu stwórzmy tabelę:

```
create table InternalEvents
(
    Id int AUTO_INCREMENT primary key,
    Title varchar(64) not null,
    Link varchar(64) not null,
    IsPublic bit not null,
    IsCancelled bit not null,
    EventDateTime datetime not null,
    PublishDateTime datetime not null,
    ShortDescription varchar(128) not null,
    ContentHTML text not null,
    MetaDescription text,
    MetaTags text,
    CreationDateTime datetime not null,
    EditDateTime datetime not null,
    Notes text,
    IsActive bit not null
);
```

Kolumna *IsActive* informuje czy obiekt jest aktywny czy też nie (funkcjonalność kosza i „usuwania” Modelu). W naszym systemie każda tabela będzie zawierać takie kolumny:

- Id
- Title
- CreationDateTime
- EditDateTime
- Notes
- IsActive

Baza danych gotowa! Teraz klasa abstrakcyjna. Pamiętajmy, że dla każdej klasy powinniśmy stworzyć osobny plik zawierający tylko jej kod. Stworzmy więc plik *Page.php* i napiszmy definicję klasy abstrakcyjnej.

```
1 <?php
2 abstract class Page
3 {
```

Nasza strona w podstawowej formie będzie mogła obsłużyć 6 akcji:

- Wyświetlenie wszystkich Modeli (rekordów z tabeli bazy danych)
- Wyświetlenie widoku tworzenia nowego Modelu (tylko wyświetlenie widoku – nie tworzenie nowego rekordu w bazie danych)
- Tworzenie nowego Modelu (rekordu w bazie danych). Później zostaną wyświetlone wszystkie Modele
- Wyświetlenie widoku edycji Modelu
- Edycja Modelu (aktualizacja bazy danych). Później zostaną wyświetlone wszystkie Modele
- Usuwanie modelu – technicznie rzecz biorąc jedynie dezaktywowanie go w bazie danych. Później zostaną wyświetlone wszystkie Modele

Aby rozpoznać jaką akcja ma zostać wykonana będziemy używali formularza *POST*. W celu uniknięcia błędów i literówek zdefiniujemy sobie kilka *stałych*, których będziemy używali jako wartości przesyłanych pól formularza oraz przy odczytywaniu tych wartości. Zawsze lepiej zapisać pewien tekst lub wartość w osobnym polu lub stałej zamiast pisać go w kodzie „z ręki” dwa lub więcej razy. Dodajmy kod do naszej klasy.

```

4     const ACTION = "action";
5     const ADD_NEW = "addNew";
6     const EDIT = "edit";
7     const EDIT_VIEW = "editView";
8     const CREATE_VIEW = "createView";
9     const DELETE = "delete";

```

Dodajmy również dwa pola: `title` oraz `tableName`. Tytuł będziemy używać w nagłówku naszej strony natomiast nazwa tabeli będzie wykorzystywana w zapytaniach SQL. Pamiętajmy, o dodaniu getterów. Nie piszmy jednak żadnych publicznych seterów! Nie chcemy aby modyfikacja tych pól była możliwa.

```

11     private string $title;
12     private string $tableName;
13
14     public function getTitle()
15     {
16         return $this->title;
17     }
18     public function getTableName()
19     {
20         return $this->tableName;
21     }

```

Jednak na etapie klasy abstrakcyjnej nie wiemy jakie będą wartości tych pól. Dla każdej klasy dziedziczącej będą inne. Dodajmy dwie **metody abstrakcyjne**, które zwrócą nam wartości tych pól i wywołajmy je w konstruktorze.

```

23     public function __construct()
24     {
25         $this->title = $this->passTitle();
26         $this->tableName = $this->passTableName();
27     }
28
29     protected abstract function passTitle(): string;
30     protected abstract function passTableName(): string;

```

Zobaczmy teraz jak powinny wyglądać nasze widoki wszystkich Modeli oraz tworzenia nowego. Znajdują się one w materiałach w formacie **HTML**.

Te widoki mają wiele wspólnych elementów. Można podzielić je na cztery sekcje:

- **HEAD** – znaczniki HTML rozpoczynające stronę, sekcje HEAD oraz rozpoczęcie BODY
- **Nagłówek** – nagłówek strony taki sam dla wszystkich widoków zawierający nazwę strony, przyciski oraz separator
- **Zawartość główna** – różna w zależności od wybranej akcji
- **Stopka** – skrypty JS, ewentualna stopka oraz znaczniki zamykające BODY i HTML

Zacznijmy od sekcji **HEAD**. Stwórzmy metodę **protected generateHEAD**, która zwróci nam string. Otwórzmy stronę HTML, podejrzujemy kod i skopiujemy wszystko do otwarcia sekcji **BODY**. W metodzie zwrócimy cały skopiowany kod. Pamiętajmy aby cały zwracany kod zamknąć w pojedynczych cudzysłowach `'`. Jako tytuł strony użyjemy naszego pola tytuł. Poza konstruktorem korzystamy już z getterów, nie bezpośrednio z pola. Musimy też do naszego projektu dodać odpowiednie style **bootstrap**, które znajdują się w materiałach z widokami. Skopiujemy foldery **js** oraz **css** do folderu naszego projektu.

```

32     protected function generateHead() : string
33     {
34         return '<!DOCTYPE html>
35         <html>
36
37         <head>
38             <meta charset="utf-8" />
39             <meta name="viewport" content="width=device-width, initial-scale=1.0" />
40             <title>
41                 '.$this->getTitle().'
42             </title>
43             <link rel="stylesheet" href="css/bootstrap.min.css" />
44             <link
45                 href="https://fonts.googleapis.com/css?family=Material+Icons|Material+Icons+Outlined|Material+I
46                 rel="stylesheet">
47             </head>';
48     }

```

Podobnie napiszmy metodę `protected generateHeader`. Tutaj też pamiętajmy aby użyć naszego tytułu.

```

50     protected function generateHeader() : string
51     {
52         return '<body>
53         <div class="container">
54             <div class="row">
55                 <div class="col-sm-12">
56                     <h1>'.$this->getTitle().'/>
57                 </div>
58             </div>
59             <div class="row">
60                 <div class="col-sm-12">
61                     <form method="POST">
62                         <button class="btn btn-primary"> Create new </button>
63                         <button class="btn btn-primary">All</button>
64                     </form>
65                 </div>
66             </div>
67         </div>
68         <hr>';
69     }

```

Teraz zatrzymajmy się na chwilę i zobaczmy jak zbudowany został formularz z przyciskami. Każdy przycisk ustawia w formularzu `POST` wartość pola akcja. Dla przycisku `Nowy` wartością powinno być `createView` natomiast dla przycisku `Wszystkie` wartość jest pusta – dlatego, że widok wszystkie jest domyślną akcją. Wcześniej stworzyliśmy stałe, aby wykorzystać je w tym właśnie formularzu. Aby uniknąć błędów użyjmy ich teraz.

```

61     <form method="POST">
62         <button name="'.self::ACTION.'" value="'.self::CREATE_VIEW.'" class="btn btn-primary"> Create new </button>
63         <button name="'.self::ACTION.'" value="" class="btn btn-primary">All</button>
64     </form>

```

Możemy też od razu wypełnić metodę zwracającą stopkę. Tutaj zwrócimy kod skryptów oraz znaczniki zamykające.

```

71     protected function generateFooter(): string
72     {
73         return '<script src="js/bootstrap.min.js"></script>
74         </body>
75         </html>';
76     }

```

Sekcja zawartości głównej może zostać wypełniona w różny sposób w zależności od wybranej akcji oraz samego `Modelu`. Aby wypełnić wszystkie możliwości dodajmy kilka metod abstrakcyjnych.

```

29     protected abstract function passTitle(): string;
30     protected abstract function passTableName(): string;
31     protected abstract function enterModelDataFromForm(): void;
32     protected abstract function generateViewAll(): string;
33     protected abstract function generateViewEdit(): string;
34     protected abstract function generateViewAdd(): string;
35     protected abstract function edit(): void;
36     protected abstract function addNew(): void;

```

Metoda `enterModelDataFromForm` będzie odpowiedzialna za przepisanie danych z formularza `POST` w widoku do `Modelu` – do tej metody jeszcze wrócimy. Metody `edit` oraz `addNew` nic nie zwracają. Zostaną one wywołane po zatwierdzeniu zmian/wypełnieniu danych nowego Modelu. Te metody przeznaczone są do wypełnienia w klasie pochodnej. Zanim przejdziemy do metody rozpoznającej polecenie brakuje nam dwóch rzeczy: metody dezaktywującej Model oraz zwracającej połączenie z bazą danych. Zaczniemy od tej drugiej. Będzie to metoda statyczna zwracająca obiekt PDO. Będziemy korzystać z bazy danych `mysql`, hostowanej `lokalnie` z użytkownikiem `root`. W domyślnej konfiguracji XAMPP MySQL nie wymaga hasła dla użytkownika `root`.

```

89     public static function openConnection(): ?PDO
90     {
91         try {
92             return new PDO("mysql:host=localhost;dbname=phpadvanced", "root");
93         } catch (PDOException $ex) {
94             echo $ex->getMessage();
95             return null;
96         }
97     }

```

Konieczne pamiętajmy o użyciu bloku `try-catch` podczas tworzenia nowego połączenia z bazą danych. Teraz możemy napisać metodę usuwającą Model. Dzięki temu, że w naszej bazie danych każda tabela zawiera kolumnę `IsActive`, która odpowiedzialna jest za usuwanie obiektu możemy zrobić to już na poziomie klasy abstrakcyjnej. Wystarczy wykonać odpowiednia zapytanie wykorzystując pole `tableName`.

```

84     protected function delete(): void
85     {
86         if (!isset($_POST["Id"])) {
87             return;
88         }
89         $query = "UPDATE " . $this->getTableName() . " SET IsActive = 0 WHERE Id = :Id";
90         $query = self::openConnection()->prepare($query);
91         $query->bindValue(":Id", $_POST["Id"], PDO::PARAM_INT);
92         $query->execute();
93     }

```

Teraz jesteśmy już gotowi do napisania metody sterującej naszą stroną internetową. Będzie ona przechwytywać wartość pola akcja z formularza `POST` i wywoływać odpowiednie metody. Musimy też zabezpieczyć naszą aplikację na wypadek braku takiej wartości – taki wypadek zdarzy się przy pierwszym uruchomieniu. Pomocniczo możemy wywołać metodę `echo print_r($_POST);`, która wypisze nam zawartość tablicy przechowującej pola z formularza `POST` – taka metoda pozwoli nam na bieżąco monitorować działanie programu. Do rozpoznania polecenia użyjemy konstrukcji `switch` w połączeniu ze stałymi.

```

105     public function initialize(): void
106     {
107         echo print_r($_POST);
108         echo $this->generateHead();
109         echo $this->generateHeader();
110         switch ($_POST[self::ACTION] ?? "") {
111             case self::CREATE_VIEW:
112                 echo $this->generateViewAdd(); break;
113             case self::EDIT_VIEW:
114                 echo $this->generateViewEdit(); break;
115             case self::EDIT:
116                 $this->enterModelDataFromForm();
117                 $this->edit();
118                 echo $this->generateViewAll(); break;
119             case self::ADD_NEW:
120                 $this->enterModelDataFromForm();
121                 $this->addNew();
122                 echo $this->generateViewAll(); break;
123             case self::DELETE:
124                 echo $this->delete();
125                 echo $this->generateViewAll(); break;
126             default:
127                 echo $this->generateViewAll(); break;
128         }
129         echo $this->generateFooter();
130     }

```

Klasa abstrakcyjna gotowa! Aby pokazać naszą stronę internetową i efektywnie z niej korzystać wystarczy wywołać metodę `initialize`. Oczywiście musimy to zrobić używając klasy pochodnej, która wypełni metody abstrakcyjne. Przechodzimy więc do tworzenia klas do obsługi konkretnego *Modelu*. Zaczniemy od klasy, która będzie odzwierciedlać nasz Model. Z racji, że każda tabela w naszej bazie danych będzie zawierać wspólne pola, możemy stworzyć klasę `Model`, która będzie je zawierać oraz klasę dziedziczącą `InternalEvent`, która będzie zawierać resztę kolumn. Pamiętajmy aby dla każdego pola napisać getter i setter. Dodajmy również w nich konstruktory wpisujące wartości wszystkich pól.

```

1  <?php
2  class Model
3  {
4      private int $id;
5      private string $title;
6      private string $notes;
7      private bool $isActive;
8      private string $creationDateTime;
9      private string $editDateTime;

```

```

1  <?php
2  include_once("Model.php");
3
4  class InternalEvent extends Model
5  {
6      private string $link;
7      private bool $isPublic;
8      private bool $isCancelled;
9      private string $publishDateTime;
10     private string $eventDateTime;
11     private string $shortDescription;
12     private string $contentHTML;
13     private string $metaDescription;
14     private string $metaTags;

```

Teraz tworzymy klasę `InternalEventsPage`, która będzie dziedziczyć po `Page`. Będzie ona obsługiwać Model `InternalEvent`. Od razu dodajmy do niej pole `Model` – przyda nam się przy *edycji* oraz *dodawaniu*.

```

1  <?php
2  include_once("InternalEvent.php");
3  include_once("Page.php");
4  class InternalEventsPage extends Page
5  {
6      private InternalEvent $model;
7
8      public function getModel() {
9          return $this->model;
10     }
11
12     public function setModel($model) {
13         $this->model = $model;
14         return $this;
15     }
16
17     protected function passTitle(): string {
18         return "Internal events";
19     }
20
21     protected function passTableName(): string {
22         return "InternalEvents";
23     }

```

Teraz przejdźmy do metody `enterModelDataFromForm`. Musi ona wypełnić nasz Model korzystając z pól formularza `POST`. Pamiętajmy o metodach `intval` oraz `boolval`, które konwertują nam string na kolejno `int` oraz `bool`. Najlepiej też podać alternatywne wartości na wypadek błędu. Moglibyśmy tutaj nieco skrócić kod wypełniając pola wspólne dla wszystkich modeli w klasie `Page`.

```

25     protected function enterModelDataFromForm(): void
26     {
27         $this->setModel(new InternalEvent(
28             intval($_POST["Id"] ?? "0"),
29             $_POST["Title"] ?? "null",
30             $_POST["Link"] ?? "null",
31             boolval($_POST["IsPublic"] ?? "0"),
32             boolval($_POST["IsCancelled"] ?? "0"),
33             $_POST["CreationDateTime"] ?? "null",
34             $_POST["EditDateTime"] ?? "null",
35             $_POST["PublishDateTime"] ?? "null",
36             $_POST["EventDateTime"] ?? "null",
37             $_POST["ShortDescription"] ?? "null",
38             $_POST["ContentHTML"] ?? "null",
39             $_POST["MetaDescription"] ?? "null",
40             $_POST["MetaTags"] ?? "null",
41             $_POST["Notes"] ?? "null",
42             boolval($_POST["IsActive"] ?? "0")
43         ));
44     }

```

Teraz wypełnijmy metody `edit` oraz `addNew`. Pamiętajmy, że zgodnie z tym co napisaliśmy w metodzie inicjalizuj będziemy mieli już Model wypełniony danymi z formularza. Wszystko więc sprowadza się do napisania odpowiednich zapytań do bazy danych.


```

321 protected function edit(): void
322 {
323     $query = "UPDATE " . $this->getTableName() . "
324     SET
325         Title = :Title,
326         Link = :Link,
327         IsPublic = :IsPublic,
328         IsCancelled = :IsCancelled,
329         ShortDescription = :ShortDescription,
330         ContentHTML = :ContentHTML,
331         PublishDateTime = :PublishDateTime,
332         EventDateTime = :EventDateTime,
333         MetaDescription = :MetaDescription,
334         MetaTags = :MetaTags,
335         Notes = :Notes,
336         EditDateTime = CURDATE()
337     WHERE Id = :Id";
338     $query = self::openConnection()->prepare($query);
339     $query->bindValue(":Title", $this->getModel()->getTitle());
340     $query->bindValue(":Link", $this->getModel()->getLink());
341     $query->bindValue(":IsPublic", $this->getModel()->getIsPublic());
342     $query->bindValue(":IsCancelled", $this->getModel()->getIsCancelled());
343     $query->bindValue(":ShortDescription", $this->getModel()->getShortDescription());
344     $query->bindValue(":ContentHTML", $this->getModel()->getContentHTML());
345     $query->bindValue(":PublishDateTime", $this->getModel()->getPublishDateTime());
346     $query->bindValue(":EventDateTime", $this->getModel()->getEventDateTime());
347     $query->bindValue(":MetaDescription", $this->getModel()->getMetaDescription());
348     $query->bindValue(":MetaTags", $this->getModel()->getMetaTags());
349     $query->bindValue(":Notes", $this->getModel()->getNotes());
350     $query->bindValue(":Id", $this->getModel()->getId());
351     $query->execute();
352 }
353
354 protected function addNew(): void
355 {
356     $query = "INSERT INTO " . $this->getTableName() . "
357     (
358         Title, Link, IsPublic, IsCancelled, EditDateTime, PublishDateTime, EventDateTime,
359         CreationDateTime, ShortDescription, ContentHTML, MetaTags, MetaDescription, Notes, IsActive
360     )
361     values
362     (
363         :Title, :Link, :IsPublic, :IsCancelled, CURDATE(), :PublishDateTime, :EventDateTime,
364         CURDATE(), :ShortDescription, :ContentHTML, :MetaTags, :MetaDescription, :Notes, 1
365     )";
366     $query = self::openConnection()->prepare($query);
367     $query->bindValue(":Title", $this->getModel()->getTitle());
368     $query->bindValue(":Link", $this->getModel()->getLink());
369     $query->bindValue(":IsPublic", $this->getModel()->getIsPublic());
370     $query->bindValue(":IsCancelled", $this->getModel()->getIsCancelled());
371     $query->bindValue(":ShortDescription", $this->getModel()->getShortDescription());
372     $query->bindValue(":ContentHTML", $this->getModel()->getContentHTML());
373     $query->bindValue(":PublishDateTime", $this->getModel()->getPublishDateTime());
374     $query->bindValue(":EventDateTime", $this->getModel()->getEventDateTime());
375     $query->bindValue(":MetaDescription", $this->getModel()->getMetaDescription());
376     $query->bindValue(":MetaTags", $this->getModel()->getMetaTags());
377     $query->bindValue(":Notes", $this->getModel()->getNotes());
378     $query->execute();
379 }

```

Pozostało nam napisać metody zwracające widoki. Znów najpierw skopiujemy kod strony HTML i odpowiednio go zmodyfikujemy. Zaczniemy od [generateViewAdd](#). Tutaj zmian będzie najmniej. Wystarczy, że zmienimy wartości wysyłane przez przycisk analogicznie jak zrobiliśmy to w metodzie [generateHEAD](#), stworzymy formularz oraz nadamy nazwy polom formularza.


```

203 protected function generateViewAdd(): string
204 {
205     return '<div class="container">
206         <form method="POST">
207     <div class="row gy-3">
208         <div class="col-md-12 col-lg-6 col-xxl-4">
209             <div class="input-group">
210                 <label class="input-group-text">
211                     <i class="material-icons-round align-middle">label</i>
212                     Title
213                 </label>
214                 <input name="Title" class="form-control validate">
215             </div>
216         </div>
217         <div class="col-md-12 col-lg-6 col-xxl-4">
218             <div class="input-group">
219                 <label class="input-group-text">
220                     <i class="material-icons-round align-middle">link</i>
221                     Link
222                 </label>
223                 <input name="Link" class="form-control validate">
224             </div>
225         </div>
226     </div>
227     <div class="col-md-12 col-lg-6 col-xxl-4">
228         <div class="row">
229             <div class="col-auto">
230                 <label class="form-check-label">
231                     Public
232                     <i class="material-icons-round align-middle">public</i>
233                 </label>
234             </div>
235             <div class="form-switch form-check col-auto">
236                 <input name="IsPublic" class="form-check-input validate" type="checkbox">
237                 <label class="form-check-label">
238                     <i class="material-icons-round align-middle">block</i>
239                     Private
240                 </label>
241             </div>
242         </div>
243     </div>
244     <div class="col-md-12 col-lg-6 col-xxl-4">
245         <div class="row">
246             <div class="col-auto">
247                 <label class="form-check-label">
248                     Cancelled
249                     <i class="material-icons-round align-middle">cancel</i>
250                 </label>
251             </div>
252             <div class="form-switch form-check col-auto">
253                 <input name="IsCancelled" class="form-check-input validate" type="checkbox">
254                 <label class="form-check-label">
255                     <i class="material-icons-round align-middle">public</i>
256                     Active
257                 </label>
258             </div>
259         </div>
260     </div>
261     <div class="col-md-12 col-lg-6 col-xxl-4">
262         <div class="input-group">
263             <label class="input-group-text">
264                 <i class="material-icons-round palette-accent-text-color align-middle">event</i>
265                 Event date
266             </label>
267             <input name="EventDateTime" class="form-control validate" type="date">
268         </div>
269     </div>
270     <div class="col-md-12 col-lg-6 col-xxl-4">
271         <div class="input-group">
272             <label class="input-group-text">

```

```

272         <i class="material-icons-round palette-accent-text-color align-middle">today</i>
273         Publish date
274     </label>
275     <input name="PublishDateTime" class="form-control validate" type="date">
276 </div>
277 </div>
278 <div class="col-sm-12">
279     <label class="form-label">
280         <i class="material-icons-round palette-accent-text-color align-middle">description</i>
281         Short description
282     </label>
283     <textarea name="ShortDescription" class="form-control validate"></textarea>
284 </div>
285 <div class="col-sm-12">
286     <label class="form-label">
287         <i class="material-icons-round palette-accent-text-color align-middle">newspaper</i>
288         Content
289     </label>
290     <textarea name="ContentHTML" class="form-control validate"></textarea>
291 </div>
292 <div class="col-sm-12">
293     <label class="form-label">
294         <i class="material-icons-round palette-accent-text-color align-middle">feed</i>
295         Meta description
296     </label>
297     <textarea name="MetaDescription" class="form-control validate"></textarea>
298 </div>
299 <div class="col-sm-12">
300     <label class="form-label">
301         <i class="material-icons-round palette-accent-text-color align-middle">subtitles</i>
302         Meta tags
303     </label>
304     <textarea name="MetaTags" class="form-control validate"></textarea>
305 </div>
306 <div class="col-sm-12">
307     <label class="form-label">
308         <i class="material-icons-round palette-accent-text-color align-middle">notes</i>
309         Notes
310     </label>
311     <textarea name="Notes" class="form-control validate"></textarea>
312 </div>
313 <div class="col-sm-12">
314     <button name="" . self::ACTION . "" value="" . self::ADD_NEW . "" class="btn btn-primary">Create
315 </div>
316 </div>
317 </form>
318 </div>;
319 }

```

Podobnie postępujemy z widokiem edycji. Tutaj jednak trzeba wcześniej pobrać edytowany Model i w formularzu umieścić ukryte pole z Id.

```

78 protected function generateViewEdit(): string
79 {
80     $query = $this->openConnection()->prepare("SELECT * FROM " . $this->getTableName() . " WHERE Id = :Id");
81     $query->bindValue(":Id", $_POST["Id"], PDO::PARAM_INT);
82     $query->execute();
83     $model = $query->fetch();
84     $output = '<div class="container">
85         <form method="POST">

```

```

84     $output = '<div class="container">
85         <form method="POST">
86         <div class="row gy-3">
87             <input hidden name="Id" value="'. $model["Id"].' ">
88             <div class="col-md-12 col-lg-6 col-xxl-4">
89                 <div class="input-group">
90                     <label class="input-group-text">
91                         <i class="material-icons-round align-middle">label</i>
92                         Title
93                     </label>
94                     <input name="Title" value="'. $model["Title"].' " class="form-control validate">
95                 </div>
96             </div>
97             <div class="col-md-12 col-lg-6 col-xxl-4">
98                 <div class="input-group">
99                     <label class="input-group-text">
100                         <i class="material-icons-round align-middle">link</i>
101                         Link
102                     </label>
103                     <input value="'. $model["Link"].' " name="Link" class="form-control validate">
104                 </div>
105             </div>
106             <div class="col-md-12 col-lg-6 col-xxl-4">
107                 <div class="row">
108                     <div class="col-auto">
109                         <label class="form-check-label">
110                             Public
111                             <i class="material-icons-round align-middle">public</i>
112                         </label>
113                     </div>
114                     <div class="form-switch form-check col-auto">
115                         <input name="IsPublic" '(. $model["IsPublic"] ? "checked" : '')' class="form-check-input">
116                         <label class="form-check-label">
117                             <i class="material-icons-round align-middle">block</i>
118                             Private
119                         </label>
120                     </div>
121                 </div>
122             </div>
123             <div class="col-md-12 col-lg-6 col-xxl-4">
124                 <div class="row">
125                     <div class="col-auto">
126                         <label class="form-check-label">
127                             Cancelled
128                             <i class="material-icons-round align-middle">cancel</i>
129                         </label>
130                     </div>
131                     <div class="form-switch form-check col-auto">
132                         <input name="IsCancelled" '(. $model["IsCancelled"] ? "checked" : '')' class="form-check-input">
133                         <label class="form-check-label">
134                             <i class="material-icons-round align-middle">public</i>
135                             Active
136                         </label>
137                     </div>
138                 </div>
139             </div>
140             <div class="col-md-12 col-lg-6 col-xxl-4">
141                 <div class="input-group">
142                     <label class="input-group-text">
143                         <i class="material-icons-round palette-accent-text-color align-middle">event</i>
144                         Event date
145                     </label>
146                     <input name="EventDateTime" value="'. date("Y-m-d", strtotime($model["EventDateTime"])).' " class="form-control validate">
147                 </div>
148             </div>
149             <div class="col-md-12 col-lg-6 col-xxl-4">
150                 <div class="input-group">
151                     <label class="input-group-text">
152                         <i class="material-icons-round palette-accent-text-color align-middle">today</i>
153                         Publish date
154                     </label>

```

```

155         <input name="PublishDateTime" value="'.date("Y-m-d", strtotime($model["PublishDateTime"])).'
156     </div>
157 </div>
158 <div class="col-sm-12">
159     <label class="form-label">
160         <i class="material-icons-round palette-accent-text-color align-middle">description</i>
161         Short description
162     </label>
163     <textarea name="ShortDescription" class="form-control validate">'.$model["ShortDescription"].'</
164 </div>
165 <div class="col-sm-12">
166     <label class="form-label">
167         <i class="material-icons-round palette-accent-text-color align-middle">newspaper</i>
168         Content
169     </label>
170     <textarea name="ContentHTML" class="form-control validate">'.$model["ContentHTML"].'</textarea>
171 </div>
172 <div class="col-sm-12">
173     <label class="form-label">
174         <i class="material-icons-round palette-accent-text-color align-middle">feed</i>
175         Meta description
176     </label>
177     <textarea name="MetaDescription" class="form-control validate">'.$model["MetaDescription"].'</te
178 </div>
179 <div class="col-sm-12">
180     <label class="form-label">
181         <i class="material-icons-round palette-accent-text-color align-middle">subtitles</i>
182         Meta tags
183     </label>
184     <textarea name="MetaTags" class="form-control validate">'.$model["MetaTags"].'</textarea>
185 </div>
186 <div class="col-sm-12">
187     <label class="form-label">
188         <i class="material-icons-round palette-accent-text-color align-middle">notes</i>
189         Notes
190     </label>
191     <textarea name="Notes" class="form-control validate">'.$model["Notes"].'</textarea>
192 </div>
193 <div class="col-sm-12">
194     <button name="" . self::ACTION . ' ' value="" . self::ADD_NEW . ' ' class="btn btn-primary">Edit <
195 </div>
196 </div>
197 </form>
198 </div>';
199 $query->closeCursor();
200 return $output;
201 }

```

Na sam koniec pozostał widok wszystkich modeli. Najpierw musimy pobrać je z bazy danych, następnie dla każdego zwrócić kartę.

```
49 protected function generateViewAll(): string
50 {
51     $query = $this->openConnection()->
52         query("SELECT * FROM " . $this->getTableName() . " WHERE IsActive = 1");
53     $output = '<div class="container">
54     <div class="row gy-3">';
55     foreach($query as $model) {
56         $output .= '<div class="col-sm-12 col-md-6 col-lg-4">
57         <div class="card">
58             <div class="card-body">
59                 <p class="card-title h5">'.$model["Title"].'</p>
60                 <p><strong>'.$model["ShortDescription"].'</strong></p>
61                 '.$model["ContentHTML"].'
62             </div>
63             <div class="card-footer">
64                 <form method="POST">
65                     <input hidden name="Id" value="'.$model["Id"].'">
66                     <button name="'.self::ACTION.'" value="'.self::EDIT_VIEW.'" class="btn btn-primary">Edit</bu
67                     <button name="'.self::ACTION.'" value="'.self::DELETE.'" class="btn btn-danger">Delete</butt
68                 </form>
69             </div>
70         </div>
71     </div>';
72     }
73     $output .= "</div></div>";
74     $query->closeCursor();
75     return $output;
76 }
```

Wszystko gotowe, w klasie [index.php](#) wystarczy wywołać metodę [inicjalizuj](#) z obiektu klasy [InternalEventsPage.php](#).